

EN3150 – Assignment 03

Simple Convolutional neural network to perform classification

Team ByteBandits

230487D	Lasan Perera
230051L	Isitha Dinujaya
230492M	Dulana Pitiwaduge
230502X	Deneth Priyadarshana

GitHub: `https://github.com/isithadinujaya/Pattern-Recognition-Assignment`

1. Question 1: Data Preparation

We used the Rock-Paper-Scissors dataset from Kaggle (`drgfreeman/rockpaperscissors`): 2,188 RGB photos of hand gestures on a green background, with 712 paper, 726 rock and 750 scissors images. Every image was resized from 300×200 to 64×64 px and split 70/15/15 with a fixed seed, giving 1,531 training, 328 validation and 329 test images. The Kaggle folder also has a duplicate copy of every image (`rps-cv-images`), so we load only the three class folders. For the pretrained models (Question 5), the same images are also normalised with the ImageNet mean and standard deviation.

```
root = Path(kagglehub.dataset_download("drgfreeman/rockpaperscissors"))

class RockPaperScissors(datasets.ImageFolder):
    def find_classes(self, directory):        # skip the rps-cv-images duplicate
        classes = ["paper", "rock", "scissors"]
        return classes, {name: i for i, name in enumerate(classes)}

transform = transforms.Compose([transforms.Resize((64, 64), antialias=True),
                                transforms.ToTensor()])
dataset = RockPaperScissors(root=root, transform=transform)      # 2188 images

total = len(dataset)
train_size, val_size = int(0.7 * total), int(0.15 * total)       # 1531, 328
test_size = total - train_size - val_size                        # 329
trainset, valset, testset = random_split(dataset, [train_size, val_size, test_size],
                                         generator=torch.Generator().manual_seed(42))
```

2. Question 2: Custom Architecture Design

Model A uses standard 5×5 convolutions. Model B replaces each one with a depthwise separable block: a 5×5 depthwise convolution followed by a 1×1 pointwise one [1]. We kept both models close to 100k parameters. Model A uses 5 feature maps and a wide dense layer (117 units), while Model B spends its cheaper convolutions on 16 feature maps and uses a smaller dense layer (36 units).

Parameter count

$$\text{Model A:}\quad \underbrace{(25\cdot 3+1)\cdot 6}_{456}+\underbrace{(25\cdot 6+1)\cdot 5}_{755}+\underbrace{(845+1)\cdot 117}_{98{,}982}+\underbrace{(117+1)\cdot 3}_{354}=\mathbf{100{,}547}$$

$$\text{Model B:}\quad \underbrace{25\cdot 3}_{75}+\underbrace{(3+1)\cdot 6}_{24}+\underbrace{25\cdot 6}_{150}+\underbrace{(6+1)\cdot 16}_{112}+\underbrace{(2704+1)\cdot 36}_{97{,}380}+\underbrace{(36+1)\cdot 3}_{111}=\mathbf{97{,}852}<100{,}000$$

These totals match the counts printed by our code. FC1 holds 98–99 % of the parameters in both models, so the depthwise separable layers barely change the model size. They do cut the computation by 3.7× (2.23 M → 0.60 M MACs). At the same width (6 and 16 filters), standard convolutions would need 6.5× more MACs than the depthwise separable ones.

Why ReLU?

ReLU is just $\max(0, x)$: one comparison per value and no exponentials like sigmoid or tanh, so it is cheap on a microcontroller and easy to run in int8. We don't put a ReLU between the depthwise and pointwise layers (as in Xception [4]), which also makes the depthwise bias unnecessary. The models

Table 1: Layer details (input $3 \times 64 \times 64$, stride 1, no padding).

Layer	Configuration	Output shape	Parameters	MACs
Model A – standard CNN				
Conv1 + ReLU	6 filters, 5×5	$6 \times 60 \times 60$	456	1,620,000
MaxPool	2×2	$6 \times 30 \times 30$	0	–
Conv2 + ReLU	5 filters, 5×5	$5 \times 26 \times 26$	755	507,000
MaxPool + Flatten	2×2	845	0	–
FC1 + ReLU	117 units	117	98,982	98,865
FC2	3 units	3	354	351
Total			**100,547**	**2,226,216**
Model B – depthwise separable CNN				
DW-Conv1	5×5, groups $= 3$, no bias	$3 \times 60 \times 60$	75	270,000
PW-Conv1 + ReLU	6 filters, 1×1	$6 \times 60 \times 60$	24	64,800
MaxPool	2×2	$6 \times 30 \times 30$	0	–
DW-Conv2	5×5, groups $= 6$, no bias	$6 \times 26 \times 26$	150	101,400
PW-Conv2 + ReLU	16 filters, 1×1	$16 \times 26 \times 26$	112	64,896
MaxPool + Flatten	2×2	2,704	0	–
FC1 + ReLU	36 units	36	97,380	97,344
FC2	3 units	3	111	108
Total			**97,852**	**598,548**

output raw scores; softmax is used only inside the loss, and on the device we simply pick the highest score, so inference needs no exponentials at all.

```
# Model A (CNN1): standard convolutions
self.conv1 = nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5)
self.conv2 = nn.Conv2d(6, 5, 5)
self.fc1 = nn.Linear(5*13*13, 117)
self.fc2 = nn.Linear(117, 3)

# Model B (CNN2): depthwise (groups = channels) + pointwise (1x1)
self.conv1 = nn.Sequential(nn.Conv2d(3, 3, 5, groups=3, bias=False), nn.Conv2d(3, 6, 1))
self.conv2 = nn.Sequential(nn.Conv2d(6, 6, 5, groups=6, bias=False), nn.Conv2d(6, 16, 1))
self.fc1 = nn.Linear(16*13*13, 36)
self.fc2 = nn.Linear(36, 3)

# forward() for both: pool(relu(conv1)) -> pool(relu(conv2)) -> flatten -> relu(fc1) -> fc2
```

3. Question 3: Optimizer Selection & Tuning

We chose **Adam with lr = 0.001**. Adam adjusts the step size for each weight separately, which helps because our layers are very different in size (75 weights in the first depthwise layer against about 97k in FC1), and the default learning rate worked without tuning [5]. We compared it with plain SGD and SGD with momentum ($\mu = 0.9$), using the same learning rate, the same starting weights and 20 epochs for every run.

Table 2: Optimizer comparison (20 epochs, lr = 0.001).

Model	Optimizer	Final train loss	Best val loss	Test accuracy
Model A	Adam	0.0622	0.1082	97.26 %
	SGD	1.0958	1.0967	31.91 %
	SGD + momentum	0.4685	0.4692	86.63 %
Model B	Adam	0.1324	0.2364	94.22 %
	SGD	1.0977	1.0984	31.91 %
	SGD + momentum	1.0979	1.0980	31.91 %

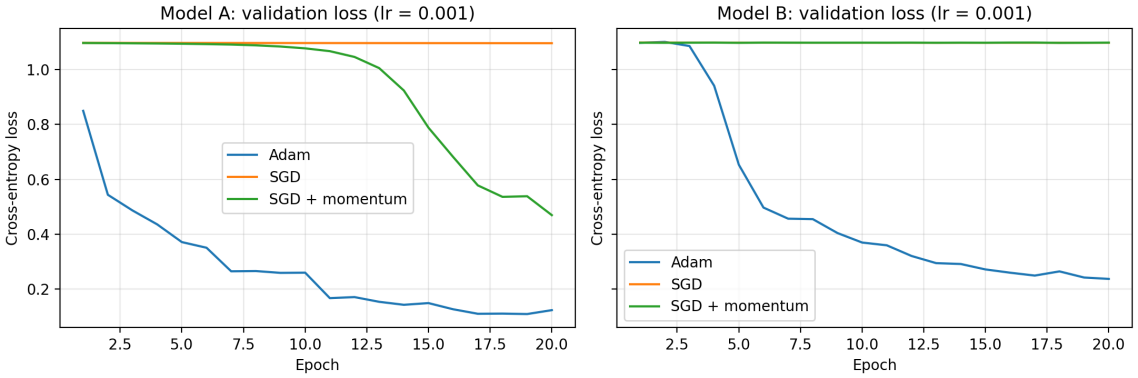


Figure 1: Validation loss for the three optimizers (lr = 0.001).

Effect of momentum

Momentum keeps a running sum of past gradients: $v_{t+1} = \mu v_t + g_t$, $\theta_{t+1} = \theta_t - \eta v_{t+1}$. When the gradient keeps pointing the same way, the step grows up to $1/(1-\mu) = 10\times$ larger, and directions that zig-zag cancel out. Training therefore moves faster without raising the learning rate.

What we observed

Plain SGD didn't learn anything in 20 epochs. The loss stayed at $\ln 3 \approx 1.099$ (equal guessing), and both models predicted *scissors* for every image, which gives 31.91 % (the 105 scissors images out of 329). Momentum got Model A out of this flat start after about 12 epochs (86.63 %, and still improving), but Model B stayed stuck. Adam got going straight away for Model A and after 3 epochs for Model B. The reason is that Adam's step size doesn't shrink when the gradients are tiny, while SGD's does. SGD usually needs a larger learning rate (0.01–0.1), so this compares the optimizers at the same learning rate rather than each at its best.

4. Question 4: Custom Model Training & Evaluation

Both models were trained with Adam for 20 epochs, keeping the weights from the epoch with the lowest validation loss. Training ran on Google Colab with an NVIDIA T4 GPU.

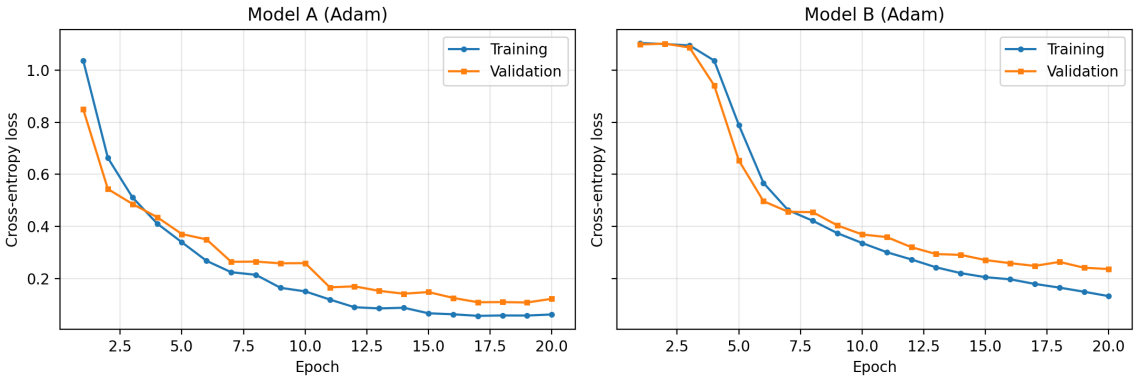


Figure 2: Training and validation loss (Adam, 20 epochs).

Model A's validation loss reaches its lowest point (0.108) at epoch 19, with a small gap to the training loss by the end, so it is just starting to overfit. Model B stays flat for the first 3 epochs and then drops quickly. Its best loss (0.236) comes at the last epoch, so it would probably improve with more training.

Model A made 9 mistakes, 8 of them on paper images. Model B made 19, most often calling rock *scissors* (6 times).

Table 3: Test results and comparison of Model A and Model B.

	Model A	Model B
Test accuracy	97.26 %	94.22 %
Macro precision	97.19 %	94.15 %
Macro recall	97.53 %	94.27 %
Macro F1-score	97.31 %	94.14 %
Parameters	100,547	97,852
Size on disk (estimated, float32)	402.2 KB	391.4 KB
Size on disk (measured `.pth`)	405.4 KB	395.3 KB
MACs per image	2.23 M	0.60 M
Training time per epoch	4.60 s	4.39 s

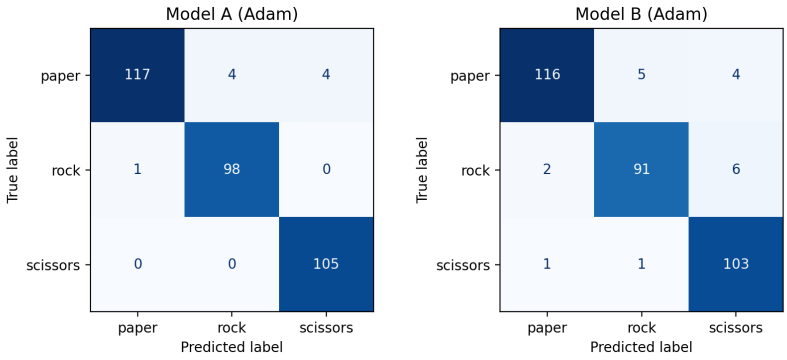


Figure 3: Confusion matrices on the test set (rows: true class, columns: predicted class).

Trade-offs

- **Computation:** Model B needs 3.7× fewer MACs. This is the main reason to use depthwise separable layers.
- **Size:** almost unchanged (−2.7 %), because the dense layer holds most of the weights.
- **Accuracy:** about 3 points lower (94.22 % vs. 97.26 %), and Model B was slower and harder to train.
- **Measured time:** only 5 % faster per epoch (4.39 s vs. 4.60 s). Most of that time goes into loading images: even MobileNetV2, with 41× Model B's MACs, takes just 5.69 s. On a microcontroller, MACs are a better guide to speed and energy use.

5. Question 5: SOTA Fine-Tuning & Evaluation

We fine-tuned **MobileNetV2** [2] and **ShuffleNetV2 ×0.5** [3], both pretrained on ImageNet and both built from depthwise convolutions. We replaced the final layer with a 3-class layer and trained all layers with Adam for 15 epochs on the same split. The learning rate was 10^{-4}, lower than for our own models so the pretrained features aren't wiped out.

```
mobilenet = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)
mobilenet.classifier[1] = nn.Linear(mobilenet.classifier[1].in_features, num_classes)  # 1280 -> 3

shufflenet = models.shufflenet_v2_x0_5(weights=models.ShuffleNet_V2_X0_5_Weights.DEFAULT)
shufflenet.fc = nn.Linear(shufflenet.fc.in_features, num_classes)                       # 1024 -> 3

mobilenet, hist_mobilenet = train_model(mobilenet, optim.Adam(mobilenet.parameters(), lr=1e-4),
                                        training_data_norm, val_data_norm, epochs=15)
# same for shufflenet
```

Both networks classified all 329 test images correctly. Zero errors on 329 images still allows a true error rate of up to about 1 % (rule of three), so this mainly shows that the task is easy for pretrained features.

Table 4: Fine-tuned pretrained models on the test set.

	MobileNetV2	ShuffleNetV2 ×0.5
Parameters	2,227,715	344,867
Size on disk	9.14 MB	1.51 MB
MACs per image	24.45 M	3.22 M
Training time per epoch	5.69 s	5.15 s
Accuracy, precision, recall, F1	100 % (all)	100 % (all)

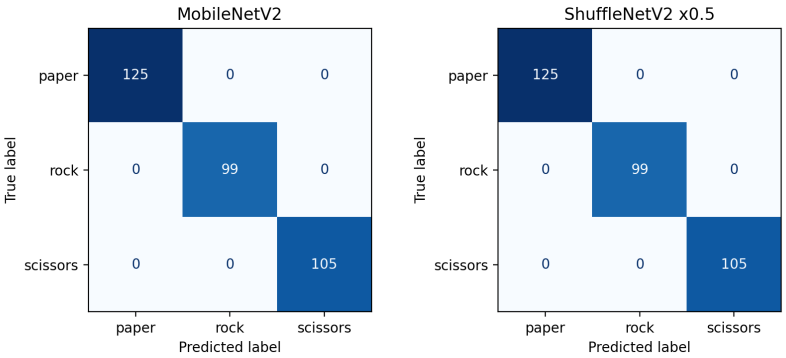


Figure 4: Confusion matrices of the fine-tuned models.

6. Question 6: Final Comparison & Trade-off Discussion

Table 5: Model B against the pretrained models.

	Model B	ShuffleNetV2 ×0.5	MobileNetV2
Parameters	97,852	344,867 (3.5×)	2,227,715 (23×)
Size on disk	395 KB	1.51 MB	9.14 MB
Size in int8 (estimate)	98 KB	345 KB	2.23 MB
MACs per image	0.60 M	3.22 M (5.4×)	24.45 M (41×)
Training time per epoch	4.39 s	5.15 s	5.69 s
Test accuracy	94.22 %	100 %	100 %

- **Accuracy:** the pretrained models reach 100 %, against 94.22 % for Model B. Features learned on ImageNet help a lot when we only have 1,531 training images.
- **Memory:** Model B is 395 KB (about 98 KB in int8), small enough for a microcontroller. ShuffleNetV2 (1.51 MB) and MobileNetV2 (9.14 MB) are too big for most microcontrollers but fine on a Raspberry Pi.
- **Computation:** ShuffleNetV2 needs 5.4× and MobileNetV2 41× more MACs than Model B, which means slower inference and more battery use on a small device.

A custom model gives full control over size and uses only simple layers that every embedded framework supports. On the other hand, it has less capacity, needs careful training (only Adam worked well), and spends 99.5 % of its weights on one dense layer. Pretrained models give top accuracy with little effort, but they're designed for 224×224 images and 1,000 classes, so they are oversized for this task.

Conclusion. For a microcontroller, Model B is the practical choice, or Model A if 3 % more accuracy is worth 3.7× the computation. For a Raspberry Pi, ShuffleNetV2 ×0.5 gives the best balance: the same 100 % as MobileNetV2 with 6.5× fewer parameters and 7.6× fewer MACs. Model B could still be improved with global average pooling instead of the big dense layer, batch normalisation, data augmentation, more epochs and int8 quantisation.

References

[1] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[2] M. Sandler et al., "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. CVPR*, 2018.

[3] N. Ma et al., "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proc. ECCV*, 2018.

[4] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. CVPR*, 2017.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.